Mandatory Practical Work 03 – 13/05/2026
Image Captioning

Deadline : Wednesday, 3 June 2026

1 General Information

The *Mandatory Practical Works* are intended to be completed in groups of up to three students.

The goal is to apply the concepts seen in the lectures and practical sessions, and to document your experimentation results in a report.

Submission Format :

- One PDF report : The report should be concise, well-structured, and reproducible and should include details such as the most relevant hyperparameters, preprocessing steps, model configurations, and evaluation settings.
- One Jupyter notebook containing the full implementation, experiments, and results.

Clearly state the **name** and **surname** of all group members at the beginning of the report and notebook.

Students are expected to implement the core model architectures themselves using PyTorch. Using pretrained CNN encoders from torchvision is encouraged. Reusing small utility functions or training infrastructure is acceptable, but using fully prebuilt image captioning systems or external implementations of the required architectures is not permitted.

Discussion between groups is encouraged, but submitted implementations and reports must represent your group,Äôs own work. All external sources and code references must be properly acknowledged.

2 Objective

The goal of this assignment is to develop and evaluate an **image captioning system**. Given an input image, the model should automatically generate a meaningful natural-language description of the image content.

3 Task Description

You are provided with a dataset containing images together with corresponding human-written captions. Your task is to design, implement, train, and evaluate models that can generate captions for previously unseen images.

The assignment consists of the following parts :

- Data exploration and preprocessing
- Implementation and training of image captioning models
- Quantitative evaluation using suitable metrics
- Comparison of different model architectures
- Qualitative analysis of generated captions

More detailed instructions and implementation guidance are provided in the accompanying Jupyter notebook.

4 Starter Notebook

A starter notebook named `captioning_starter.ipynb` is provided.

The notebook contains :

- instructions for setting up the environment,
- data preparation and loading utilities,
- examples for visualizing images and captions,
- helper code for preprocessing and experimentation.

You are expected to extend this notebook with your own implementation, experiments, and analysis. The final notebook should contain all code required to reproduce your results.

5 Required Model Architectures

You must implement, train, and evaluate the following two architectures.

Baseline Model

Implement a classical encoder-decoder image captioning model consisting of :

- a pretrained CNN encoder (e.g. ResNet18),
- an LSTM-based decoder.

As reference, consult :

> O. Vinyals et al., *Show and Tell : A Neural Image Caption Generator*, 2015.
> https://arxiv.org/pdf/1411.4555

Attention-Based Model

Implement an attention-based image captioning architecture consisting of :

— a pretrained CNN encoder,

— an LSTM decoder,

— an attention mechanism over spatial image features.

The attention module should allow the decoder to focus on spatially localized image features during caption generation.

As reference, consult :

> K. Xu et al., *Show, Attend and Tell : Neural Image Caption Generation with Visual Attention*, 2016.
> https://arxiv.org/pdf/1502.03044

Comparison

Both architectures must be trained and compared using the same dataset and evaluation setup. Show the training / validation curves for the loss and possibly also the BLEU scores (over epochs).

Discuss the advantages and limitations of each approach.

6 Training Requirements

When implementing and training your models, consider the following aspects carefully :

— **Data augmentation :** Use suitable augmentations for the training images. The dataset is relatively small, therefore augmentation is important for improving generalization.

— **Shared encoder setup :** Use the same pretrained image encoder for both architectures. For the attention-based model, preserve the spatial feature maps required by the attention mechanism (e.g. for ResNet18, use the final convolutional feature tensor of size $512 \times 7 \times 7$).

— **Teacher forcing :** Use teacher forcing during training. Since training and inference differ in this setup, implement :

  — a `forward()` method used during training,

  — a `generate()` method used during inference and evaluation.

  Teacher forcing is essential for stable and efficient training of sequence generation models !

— **Inference strategy :** Use greedy decoding for caption generation, i.e. at each timestep select the token with the highest predicted probability.

— **Loss function and masking :** Design the loss carefully and properly handle padding and end-of-sequence tokens.

— **Regularization and overfitting control :** Apply appropriate techniques such as dropout, weight decay, early stopping, or augmentation.

— **Experiment tracking :** Use an MLOps or experiment tracking framework such as Weights & Biases (W&B) to monitor training progress and compare experiments.

7 Evaluation

Evaluate your models using suitable quantitative metrics. At minimum, include :

— Perplexity
— corpus-level BLEU-1 to BLEU-4

In addition to quantitative evaluation, include a qualitative analysis of generated captions using representative examples.

Your qualitative analysis should include both successful and failure cases. Discuss typical captioning errors such as :

— missing objects,
— hallucinated objects,
— repetitive text,
— incorrect relationships,
— or overly generic captions.

Finally, include an analysis of failure cases and discuss possible reasons for these failures.

8 Additional Tasks

In addition to the required architectures, address **at least two** of the following extensions :

— **Beam Search** : Easy
Implement beam search decoding and compare it with greedy decoding.
— **Attention Visualization** : Easy
Visualize attention maps by overlaying attention heatmaps on the original image for selected generated words for generated words. Analyze whether the model attends to meaningful image regions.
— **Pretrained Word Embeddings** : Moderate
Investigate the use of pretrained embeddings such as GloVe or Word2Vec.
— **Alternative Attention Mechanisms** : Moderate
Compare Bahdanau attention with dot-product attention as commonly used in transformer architectures.
— **Transformer-Based Decoder** : Hard
Implement a transformer decoder with cross-attention to image features and compare it to the LSTM-based attention model.

Given the difficulty of the problem you will gain additional bonus points.

9 Expected Deliverables

Submit the following :

— **Jupyter Notebook** A notebook containing your complete implementation, experiments, visualizations, and results.

Naming convention :

`captioning_<groupname>.ipynb` and

Example :

`captioning_henneberg_melchior.ipynb`

Before submission, ensure that all notebook cells execute without errors, outputs are included and unnecessary intermediate cells or files are removed.

— **Report (PDF)** A report containing (max 4 pages) :

- description of your approach, including preprocessing choices,
- architectural and implementation choices,
- experimental setup,
- quantitative results,
- qualitative analysis,
- discussion of limitations and possible improvements.

Naming convention :

`captioning_<groupname>.pdf` and

Only one submission per group is required.

10 Evaluation Criteria

Your submission will be evaluated based on :

- correctness and completeness of the implementation,
- quality of the generated captions,
- appropriateness of the evaluation methodology,
- quality of experiments and comparisons,
- clarity and depth of the report and analysis.

Note that correct experimentation methodology and analysis are considered more important than achieving the highest metric scores. Given the limited dataset only limited performance is expected !